ENGINEERING ASSESSMENT PORTFOLIO | CYNBIT

Revio InsightOps AI

Technical Approach & Implementation Report

"Turn customer reviews into evidence-backed actions."

Architecture, Machine Learning Pipeline, Rule-Based Engines, and Held-Out Benchmark Evaluation

Prepared By:	Bhishant Sharma
Role / Assessment:	Cynbit AI/ML Engineer Intern Assessment
Submission Date:	October 2026
Evaluation Target:	Cynbit Technical & Hiring Review Team
Live Streamlit Demo:	Open Live Application (Streamlit Cloud)
GitHub Repository:	View Source Code (github.com/Bhishant)

1. Project Information & Technical Endpoints

Revio InsightOps AI is an end-to-end customer feedback intelligence system that combines classical machine learning with transparent heuristic rule engines to classify sentiment, detect operational defect categories, and prioritize high-urgency customer complaints. This report provides an exhaustive technical specification of the system architecture, feature extraction pipelines, classification algorithms, rule engine mechanics, benchmark evaluation results, and codebase structure.

Technical Specification	System Attribute / Repository Detail	Engineering Notes
System Name	Revio InsightOps AI	Assessment MVP prototype
Primary Developer	Bhishant Sharma	AI/ML Engineer Intern Candidate
Assessment Context	Cynbit Technical Assessment Portfolio	Focus on practical ML engineering and architecture
Public Web Application	https://revio-insightops-ai.streamlit.app/	Clickable link; hosted on Streamlit Cloud
Version Control Repository	https://github.com/Bhishant/revio-insightops-ai	Git repository containing codebase and model scripts
Benchmark Training Dataset	Yelp Restaurant Reviews Sentiment Dataset	14,350 curated, deduplicated records (Zenodo 18723813)
Core Pipeline	TF-IDF Vectorizer + Logistic Regression	scikit-learn pipeline serialized via joblib
Issue & Priority Engines	Deterministic Regex & Transparent Heuristics	Auditable rule engines in src/core.py

- **Live Deployed Streamlit Application:** https://revio-insightops-ai.streamlit.app/
- **GitHub Source Code Repository:** https://github.com/Bhishant/revio-insightops-ai

> **[SYSTEM ARCHITECTURE TRANSPARENCY]** ENGINEERING INTEGRITY NOTE: This document provides complete technical transparency. The sentiment prediction subsystem utilizes an n-gram TF-IDF vectorizer paired with a linear Logistic Regression classifier. The system does not claim or use transformer architectures, Large Language Models (LLMs), BERT, or deep neural networks. Issue categorization and priority scoring are separate, auditable rule-based engines, not trained multi-class classifiers.

2. Executive Summary

Revio InsightOps AI was engineered to demonstrate how a practical, computationally efficient, and transparent AI system can be designed to solve unstructured text classification challenges in enterprise

operations. Rather than defaulting to resource-heavy generative models or black-box neural networks, the system implements a decoupled, deterministic pipeline optimized for high throughput, zero external API leakage, and complete auditability.

The technical architecture consists of three interconnected subsystems: (1) A supervised text classification pipeline utilizing sublinear TF-IDF n-gram feature extraction (100,000 max features) and balanced L2-regularized Logistic Regression to infer sentiment polarity; (2) A deterministic regex-based issue categorization engine that maps customer comments across five operational domains (Delivery, Product quality, Customer support, Billing & payment, and App & website); and (3) A transparent heuristic priority scoring engine that calculates urgency based on sentiment polarity and explicit grievance keywords, strictly bounded between 1 and 55.

Trained on 11,480 Yelp customer review records and evaluated against a held-out test split of 2,870 records, the sentiment pipeline achieved 81.85% overall accuracy and a macro F1-score of 0.6951. The model demonstrates high fidelity on positive sentiment (0.906 F1; 0.913 precision; 0.899 recall) and strong operational efficacy on negative sentiment (0.785 F1; 0.766 precision; 0.806 recall), successfully surfacing 80.6% of customer complaints. In a live deployment test processing 14,351 uploaded customer reviews, the application completed inference in seconds on standard CPU hardware, identifying 2,799 negative reviews and isolating 76 high-priority operational escalations.

3. Introduction and Engineering Objectives

The engineering goal of Revio InsightOps AI was to develop a production-viable Minimum Viable Product (MVP) capable of processing large volumes of qualitative customer feedback into structured, actionable operational data. To satisfy real-world enterprise requirements, the architecture was guided by five core engineering objectives:

- **1. Computational Efficiency:** Eliminate the latency, expense, and nondeterministic drift of remote LLM APIs by running all feature extraction and inference locally on CPU.
- **2. End-to-End Interpretability:** Ensure that every classification and score can be audited. Model weights are linear, issue rules are regex-based, and priority formulas are mathematically bounded.
- **3. Deterministic Reproducibility:** Guarantee that identical inputs produce identical predictions. Training and evaluation pipelines use locked random seeds and serialized scikit-learn artifacts.
- **4. Data Sovereignty & Privacy:** Sensitive customer feedback is processed entirely in-memory within the active session, preventing unauthorized data exfiltration or third-party leakage.
- **5. Ergonomic Human-in-the-Loop UX:** Present intelligence through an intuitive Streamlit interface that enables non-technical operations analysts to interact with machine learning outputs seamlessly.

4. Functional Requirements

The system implements a rigorous functional specification covering the entire data lifecycle:

Functional ID	System Capability	Technical Implementation Specification	Validation Criteria
FR-01	CSV Ingestion & Validation	Validate presence of mandatory 'text' column; support optional 'rating', 'branch', 'date'.	Rejects malformed CSVs; sanitizes null and empty strings.

Functional ID	System Capability	Technical Implementation Specification	Validation Criteria
FR-02	Text Normalization	Whitespace normalization, case folding, unicode accent stripping, regex escaping.	Zero regex injection vulnerabilities; uniform casing for matching.
FR-03	Sentiment Inference	TF-IDF (100k features, n-grams 1-2) + Logistic Regression (C=2.0, balanced).	Sub-millisecond inference per row; outputs sentiment and confidence probability.
FR-04	Issue Categorization	Regex word boundary matching against 5 curated operational dictionaries.	Tallies matching phrases; assigns category with highest hits; defaults to 'Other'.
FR-05	Priority Scoring	Heuristic formula: base sentiment score + urgency (+25) + unresolved (+15); min(score, 55).	Bounded output [1, 55]; flags scores >= 40 as high-priority escalations.
FR-06	Dashboard Visualization	Streamlit UI with Plotly charts (donut sentiment, horizontal issue bars, KPI cards).	Interactive filtering; real-time reactive re-rendering.
FR-07	Data Export	Export enriched Pandas DataFrame to UTF-8 CSV ('revio_analyzed_reviews.csv').	Downloadable directly from Review Explorer tab.
FR-08	Model Governance Tab	Dynamic JSON parsing and rendering of saved held-out benchmark evaluation metrics.	Displays exact evaluation metrics without re-running training in UI.

5. Technology Stack

The project leverages a modern, stable Python data science and machine learning stack. Each component was selected based on performance, ecosystem maturity, and deployment simplicity:

Component / Library	Version Specification	Architectural Role	Engineering Justification

Component / Library	Version Specification	Architectural Role	Engineering Justification
Python	>= 3.10, < 3.14	Core runtime environment	High-performance standard library, robust typing, and cross-platform compatibility.
Streamlit	>= 1.40.0	Application framework & UI	Enables rapid, reactive web application development with minimal frontend boilerplate.
pandas	>= 2.0.0	Tabular data manipulation	Vectorized string manipulation, schema validation, and high-speed DataFrame filtering.
scikit-learn	>= 1.4.0	ML pipeline & feature extraction	Standardized Pipeline, TfidfVectorizer, LogisticRegression, and evaluation metrics.
joblib	>= 1.3.0	Model serialization	Efficient disk serialization and deserialization of the trained scikit-learn pipeline.
Plotly	>= 5.20.0	Interactive data visualization	GPU-accelerated, responsive charts (donut charts, bar charts) embedded in Streamlit.
pytest	>= 8.0.0	Automated test suite	Unit testing framework verifying rule-matching regex and priority scoring logic.

6. System Architecture

Revio InsightOps AI employs a modular, layered architecture designed to maintain a strict separation of concerns. The architecture explicitly bifurcates Machine Learning operations from deterministic rule-based heuristic calculations, ensuring that each subsystem can be tested, calibrated, and updated independently.

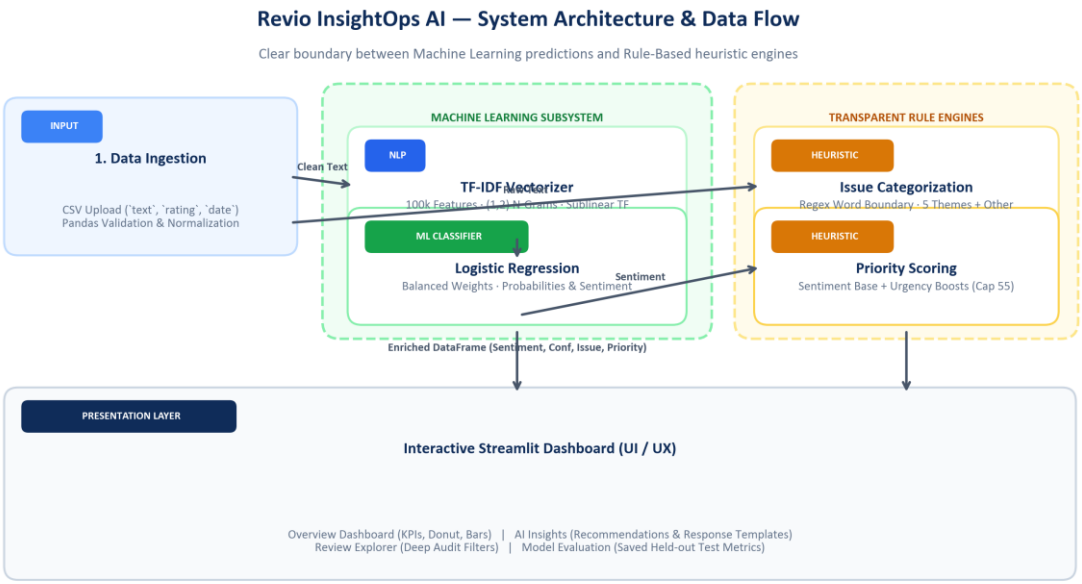


Figure 1: High-level system architecture showing separation between ML Subsystem and Transparent Rule Engines.

Architectural Layer Breakdown:

- **1. Ingestion & Validation Layer:** Handles CSV file uploads via the Streamlit sidebar, performs schema validation against required columns, removes nulls and empty strings, and formats the active Pandas DataFrame.
- **2. Machine Learning Subsystem:** Encapsulated in the scikit-learn Pipeline ('models/sentiment_pipeline.joblib'). Converts raw review strings into sparse TF-IDF vectors and executes multi-class Logistic Regression inference to predict sentiment polarity ('positive', 'neutral', 'negative') and maximum class probability.
- **3. Rule-Based Heuristic Engines:** Implemented in 'src/core.py'. Two independent functions operate on the normalized review text and predicted sentiment: 'detect_issue()' maps keyword phrases to operational domains, and 'priority_score()' computes a bounded heuristic urgency score.
- **4. Presentation & Interaction Layer:** Streamlit-based web dashboard providing four interactive tabs: Overview Dashboard, AI Insights, Review Explorer, and Model Evaluation.
- **5. Persistence & Export Layer:** Generates downloadable UTF-8 CSV files containing enriched columns ('Sentiment', 'Confidence', 'Issue', 'Priority') for enterprise downstream workflows.

7. Application Workflow and Execution Lifecycle

When an analyst accesses Revio InsightOps AI, the application executes a highly optimized execution lifecycle:

- **Model Resource Caching:** During Streamlit initialization, the serialized pipeline is loaded via '@st.cache_resource def load_model()'. This caches the 4.3 MB pipeline object in memory, ensuring that subsequent user interactions and filter adjustments incur zero deserialization overhead.
- **Reactive Ingestion:** When a user uploads a new CSV or selects the built-in sample dataset, the data is validated. Missing 'text' columns trigger immediate user-friendly alerts via 'st.error()'.

- **Vectorized Batch Inference:** The loaded DataFrame undergoes vectorized batch inference. Model predictions, probability scores, issue detection, and priority scoring run across all rows concurrently in a single execution pass.
- **Tabbed Viewport Rendering:** Streamlit tabs render lazily based on user selection, minimizing DOM overhead and ensuring smooth chart rendering even with tens of thousands of rows.

8. Data Input and Preprocessing

Robust input validation and text normalization are critical to preventing pipeline exceptions and ensuring clean feature extraction:

A. Input Schema Requirements

The application accepts arbitrary CSV files conforming to the following specification:

- • **text (Mandatory, String):** Contains the raw textual customer review. Non-string inputs are converted to strings; null values and whitespace-only rows are dropped.
- • **rating (Optional, Numeric/String):** Numerical or categorical customer score (e.g., 1 to 5 stars). Retained for review inspection and CSV export.
- • **branch (Optional, String):** Identifies physical store location, operating region, or subsidiary unit. Enables branch-level drilldowns in Review Explorer.
- • **date (Optional, String/Date):** Submission timestamp (e.g., YYYY-MM-DD). Enables temporal context during review-level inspection.

B. Text Normalization Pipeline

Text normalization is applied consistently across both the machine learning feature extraction pipeline and the rule engine. In 'src/core.py', the 'normalize_text()' function applies regex substitution to collapse consecutive whitespace characters and enforce lowercase formatting:

> **[CODE DEFINITION: src/core.py — normalize_text]** def normalize_text(text):
> """Normalize text before matching issue-related phrases."""
> return re.sub(r'\s+', ' ', str(text).lower()).strip()

9. Sentiment Analysis Model Pipeline

The sentiment prediction subsystem is structured as a scikit-learn Pipeline comprising a TF-IDF text vectorizer and a multi-class Logistic Regression classifier. This combination represents an optimal baseline for text classification:

A. TF-IDF Feature Extraction Specification

Term Frequency-Inverse Document Frequency (TF-IDF) transforms raw string inputs into a high-dimensional numerical feature matrix. The vectorizer configuration in 'train_real_yelp_model.py' is parameterized as follows:

Parameter	Configured Value	Algorithmic & Engineering Rationale

Parameter	Configured Value	Algorithmic & Engineering Rationale
lowercase	True	Normalizes all characters to lowercase to eliminate case-based feature duplication.
strip_accents	'unicode'	Converts accented characters to base ASCII, improving generalization across varied input encodings.
ngram_range	(1, 2)	Extracts both individual words (unigrams) and adjacent word pairs (bigrams), capturing vital negation phrases such as 'not good' or 'never again'.
min_df	2	Ignores terms that appear in fewer than 2 documents, pruning spelling anomalies and rare noise.
max_df	0.98	Ignores terms appearing in more than 98% of documents, filtering out ubiquitous corpus terms.
max_features	100,000	Caps the feature dictionary at the top 100,000 n-grams by frequency, constraining memory consumption.
sublinear_tf	True	Applies sublinear scaling (1 + log(tf)), dampening the effect of repetitive words in lengthy reviews.
stop_words	'english'	Filters standard English grammatical stopwords ('the', 'is', 'at') to concentrate signal on sentiment-bearing adjectives and verbs.

B. Logistic Regression Classifier Specification

The classification stage employs a multi-class Logistic Regression model trained via the L-BFGS numerical optimization algorithm:

Hyperparameter	Configured Value	Algorithmic & Engineering Rationale
solver	'lbfgs'	Limited-memory BFGS quasi-Newton optimization; handles large sparse feature spaces with fast convergence.
max_iter	1500	Provides sufficient gradient descent iterations to guarantee convergence across 100,000 features.
C	2.0	Inverse regularization strength (1/lambda = 2.0). Mildly relaxes L2 penalty to capture nuanced sentiment terms.
class_weight	'balanced'	Adjusts weights inversely proportional to class frequencies: w_j = n / (k * n_j). Penalizes misclassifications on minority classes (Neutral and Negative).

Hyperparameter	Configured Value	Algorithmic & Engineering Rationale
random_state	42	Enforces strict mathematical reproducibility across training iterations.

10. Model Training Methodology

The model training pipeline was developed in 'train_real_yelp_model.py' to provide an end-to-end reproducible training workflow:

A. Dataset Source and Usable Record Preparation

The model was trained on the Yelp Restaurant Reviews Sentiment Dataset (Zenodo record 18723813). The dataset ingestion script maps ratings into three normalized sentiment classes: 'positive', 'neutral', and 'negative'. Records with missing text, null labels, or duplicate review strings were eliminated, yielding exactly 14,350 usable records.

Dataset Partition	Row Count	Percentage	Sampling Strategy
Training Split	11,480	80.0%	Stratified random sampling (random_state=42)
Held-Out Testing Split	2,870	20.0%	Stratified random sampling (random_state=42)
Total Usable Records	14,350	100.0%	Deduplicated, verified 3-class ground truth

B. Ground-Truth Class Distribution

The 14,350 curated records exhibit natural real-world class imbalance, reflecting standard consumer review behavior where satisfied customers constitute the majority:

Sentiment Class	Curated Total Records	Training Split (80%)	Testing Split (20%)	Class Share
Positive	9,857	7,886	1,971	68.69%
Negative	2,698	2,158	540	18.80%
Neutral	1,795	1,436	359	12.51%
Total Usable Records	14,350	11,480	2,870	100.0%

11. Model Evaluation Metrics and Performance

Evaluation was conducted strictly on the 2,870 held-out test split examples. All metrics reported below reflect the serialized artifact saved at 'models/evaluation_metrics.json':

A. Overall Performance Summary

Evaluation Metric	Measured Value	Interpretation
Overall Test Accuracy	81.85% (0.818467)	Proportion of correct predictions across all 2,870 test examples.
Macro F1-Score	0.6951 (0.695086)	Unweighted average of F1-scores across all three classes, giving equal weight to minority classes.
Weighted F1-Score	0.8193 (0.819325)	Support-weighted average of F1-scores reflecting overall dataset composition.
Test Split Support	2,870 records	Independent held-out evaluation sample size.

B. Class-Wise Classification Report

Class Label	Precision	Recall	F1-Score	Test Support
Positive	0.913	0.899	0.906	1,971
Neutral	0.392	0.396	0.394	359
Negative	0.766	0.806	0.785	540
Macro Average	0.691	0.700	0.695	2,870
Weighted Average	0.820	0.818	0.819	2,870

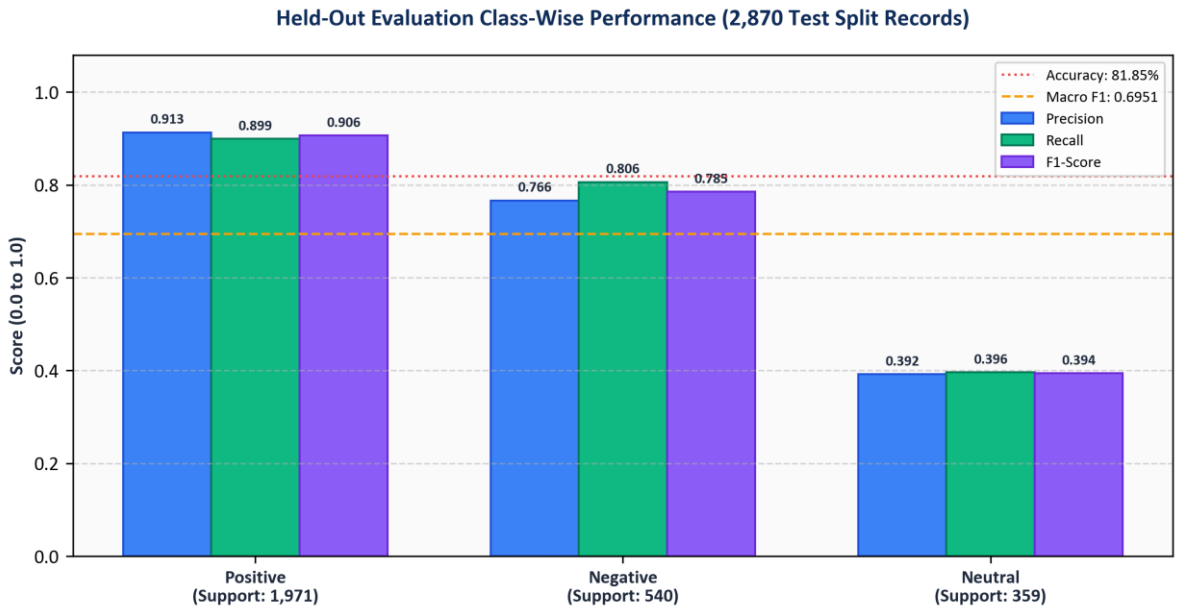


Figure 2: Precision, Recall, and F1-score across Positive, Negative, and Neutral classes on 2,870 test records.

C. Confusion Matrix Analysis

The confusion matrix illuminates specific error patterns and cross-class confusions across the 2,870 held-out examples:

True \ Predicted	Pred: Positive	Pred: Neutral	Pred: Negative	Total True Support
True Positive	1,772 (89.9%)	150 (7.6%)	49 (2.5%)	1,971
True Neutral	133 (37.0%)	142 (39.6%)	84 (23.4%)	359
True Negative	35 (6.5%)	70 (13.0%)	435 (80.6%)	540

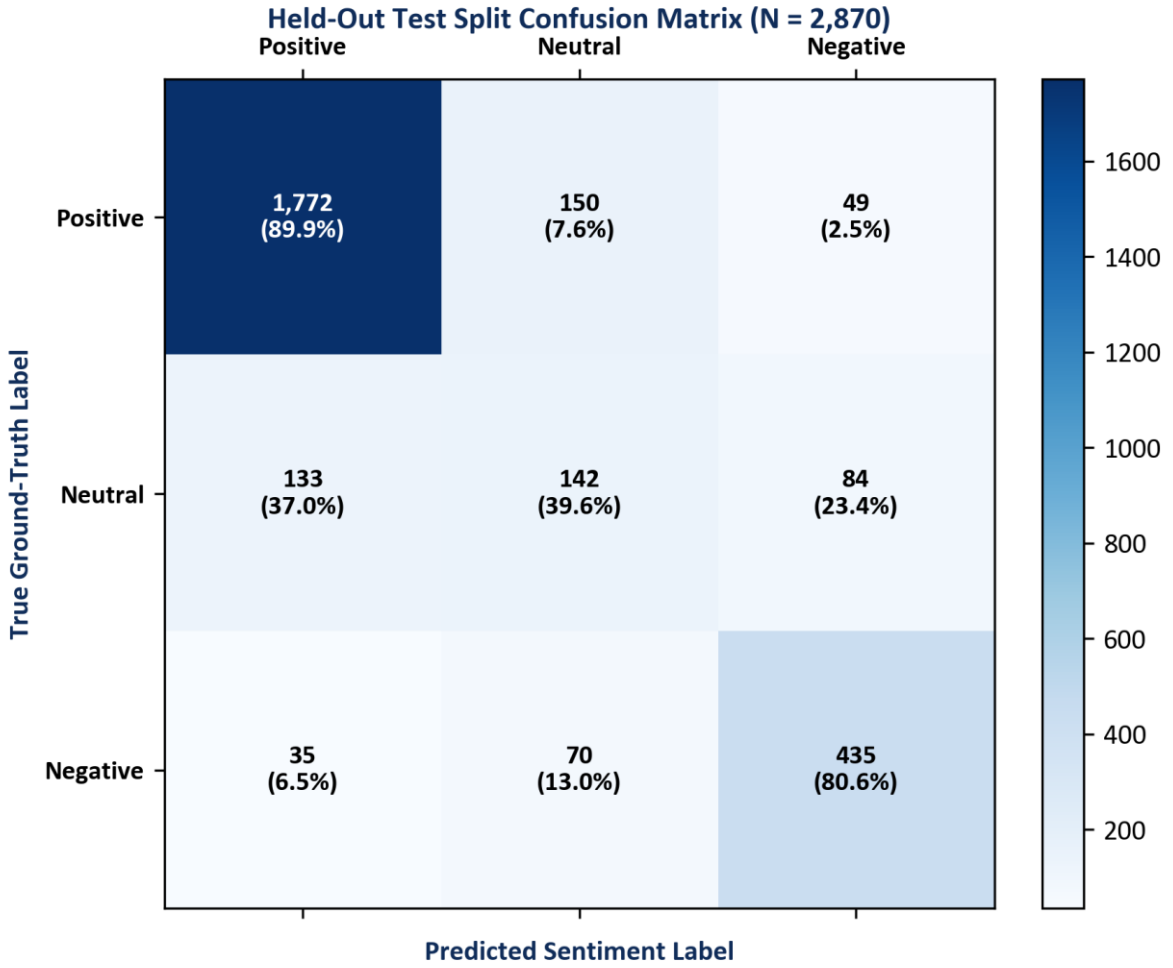


Figure 3: Confusion matrix heatmap showing true versus predicted counts and class-level accuracy rates.

12. In-Depth Class-Wise Performance Analysis

A rigorous engineering assessment requires dissecting where the model excels and where it encounters structural constraints:

- **Positive Sentiment Fidelity (F1 = 0.906):** The positive class achieves exceptional precision (0.913) and recall (0.899), yielding an F1-score of 0.906. This strong performance is driven by two factors: large training support (7,886 examples) and highly distinct positive vocabulary ('delicious', 'friendly', 'excellent', 'perfection') that creates strong linear separation in the TF-IDF feature space.
- **Negative Sentiment Triage Efficacy (F1 = 0.785):** Negative sentiment classification achieves 0.766 precision and 0.806 recall (F1 = 0.785). From an operational standpoint, high recall (80.6%) is the most critical metric: the system successfully captures 435 out of 540 actual negative complaints, ensuring that customer grievances are routed into the triage queue. Only 35 negative reviews (6.5%) were erroneously classified as positive.
- **The Neutral Sentiment Challenge (F1 = 0.394):** The neutral class exhibits lower performance across precision (0.392), recall (0.396), and F1-score (0.394). In consumer restaurant reviews, 3-star feedback rarely contains purely neutral language; instead, it consists of mixed sentiment ('food was fantastic but waiter was rude', 'average experience for the high price'). A bag-of-words linear model struggles to weigh opposing clauses without syntactic dependency parsing. Furthermore, neutral represents the smallest training class (1,436 training rows).

- **Macro F1 vs Accuracy Under Class Imbalance:** Evaluating this system purely on accuracy (81.85%) would be misleading due to positive class dominance (68.7%). The macro F1-score of 0.6951 provides an honest, unweighted evaluation of multi-class balance and highlights neutral sentiment as the primary target for future model iteration.

13. Issue Categorisation Logic

Issue categorization is implemented in 'src/core.py' using a transparent, deterministic regex rule engine. This is not a machine learning classifier; it is a keyword matching system engineered for immediate explainability and domain calibration.

Issue Category	Curated Regex Keyword Phrases	Operational Domain Focus
Delivery	'delivery', 'late delivery', 'delayed delivery', 'shipping', 'courier', 'package', 'arrived late', 'takeout', 'take out', 'pickup', 'pick up', 'waiting for order', 'order never arrived'	Courier logistics, dispatch delays, takeout/pickup fulfillment.
Product quality	'product quality', 'broken product', 'defective', 'damaged item', 'faulty', 'poor quality', 'food quality', 'food was cold', 'undercooked', 'overcooked', 'raw food', 'food poisoning', 'plastic in food', 'portion size', 'tasteless', 'bland food', 'bad food'	Core product manufacturing, kitchen execution, food safety, physical damage.
Customer support	'customer support', 'customer service', 'rude staff', 'unhelpful staff', 'ignored my complaint', 'no response', 'manager', 'staff was rude', 'poor service', 'terrible service', 'waiter', 'waitress', 'server', 'hostess', 'host'	Staff behavior, responsiveness, desk manners, supervisor escalation.
Billing & payment	'charged', 'refund', 'billing', 'payment', 'overpriced', 'overcharged', 'charged twice', 'wrong bill', 'incorrect bill', 'hidden fee', 'extra charge', 'tip', 'gratuity', 'expensive'	Transaction accuracy, payment processing, duplicate charges, pricing disputes.
App & website	'app', 'website', 'crashes', 'crashed', 'login', 'loading', 'checkout error', 'page error', 'online ordering', 'booking website'	Digital touchpoint usability, mobile app crashes, portal checkout errors.
Other	Default fallback when total phrase match score == 0	Uncategorized reviews or feedback lacking matched domain keywords.

Algorithmic Implementation

The 'detect_issue()' function normalizes the input text and evaluates regex word boundaries to prevent substring false-positives (e.g., matching 'tip' inside 'multiple'):

[CODE DEFINITION: src/core.py — detect_issue]

```
def detect_issue(text):
    lowered = normalize_text(text)
    scores = {}
    for category, phrases in ISSUE_RULES.items():
        scores[category] = sum(
            1 for phrase in phrases
            if re.search(r'(?<!\w)' + re.escape(phrase) + r'(?!\w)', lowered)
        )
    best_category = max(scores, key=scores.get)
    return best_category if scores[best_category] > 0 else 'Other'
```

14. Priority Scoring Methodology

Priority scoring provides a transparent heuristic ranking mechanism to sequence reviews for manual inspection. It is implemented in 'priority_score(sentiment, text)' in 'src/core.py'. The score is strictly bounded between 1 and 55 and combines predicted sentiment with explicit grievance phrases.

A. Mathematical Formulation

The priority score is calculated using the following bounded piecewise formula:

[FORMULA: Priority Scoring Heuristic] Priority Score = min(BaseScore(Sentiment) + UrgencyBoost + UnresolvedBoost, 55)

Where:
- BaseScore(Negative) = 15 | BaseScore(Neutral) = 5 | BaseScore(Positive) = 1 (Fallback = 5)
- UrgencyBoost = +25 if ANY urgent keyword phrase is present in review text
- UnresolvedBoost = +15 if ANY unresolved grievance phrase is present in review text
- Max Upper Bound = 55

B. Keyword Dictionaries for Score Elevation

- **Urgent Keyword List (+25):** ['refund', 'charged twice', 'fraud', 'unsafe', 'food poisoning', 'allergic reaction', 'plastic in food', 'overcharged', 'broken']. Adds +25 to priority score.
- **Unresolved Grievance List (+15):** ['ignored', 'not resolved', 'several calls', 'no response', 'still waiting', 'no refund']. Adds +15 to priority score.

Sentiment	Urgent Keyword Match	Unresolved Match	Calculated Score	Operational Triage Level
Positive	No (0)	No (0)	1 / 55	Routine / Low Priority
Neutral	No (0)	No (0)	5 / 55	Standard Monitoring

Sentiment	Urgent Keyword Match	Unresolved Match	Calculated Score	Operational Triage Level
Negative	No (0)	No (0)	15 / 55	Moderate Priority (Standard Negative)
Negative	No (0)	Yes (+15)	30 / 55	Elevated Priority (Unresolved Complaint)
Negative	Yes (+25)	No (0)	40 / 55	High Priority Flag (Immediate Attention)
Negative	Yes (+25)	Yes (+15)	55 / 55 (Capped)	Maximum Critical Escalation (High Risk)

15. Streamlit Application Implementation

The front-end user experience is implemented in 'app.py' using Streamlit. The application provides four dedicated functional modules, each targeting a specific analytical need:

Tab 1: Overview Dashboard

Renders four metric cards ('st.metric') showing total reviews, negative reviews with volume percentage, high-priority flags (score >= 40), and low-confidence predictions (< 0.55). Features two Plotly visualizations: a donut chart illustrating sentiment proportions and a horizontal bar chart ranking detected issue categories.

Tab 2: AI Insights Module

Focuses on root-cause investigation. When an analyst selects an issue category from the dropdown, the module displays the count of negative reviews in that theme, the theme's average priority score, and the maximum individual score. It quotes the top 3 highest-priority reviews, displays deterministic operational auditing recommendations via 'make_recommendation()', and presents an editable response draft via 'make_response()'.

Tab 3: Review Explorer Module

Provides granular search and inspection. Multi-select widgets allow simultaneous filtering across sentiment classes and issue categories. The reactive data grid dynamically displays Customer Review text, Predicted Sentiment, Model Confidence, Issue Category, Priority Score, and optional metadata. A download button triggers immediate export to 'revio_analyzed_reviews.csv'.

Tab 4: Model Evaluation Module

Reads 'models/evaluation_metrics.json' directly and presents the held-out test split evaluation benchmarks. Displays evaluation row count (2,870), overall accuracy (81.85%), macro F1 (0.6951), and

the complete class-wise report table, accompanied by explicit cautionary notes regarding domain transferability.

16. Deployment Architecture and GitHub

The application is continuously deployed on Streamlit Community Cloud and linked to the public GitHub repository:

- **Source Repository:** https://github.com/Bhishant/revio-insightops-ai
- **Continuous Deployment:** Commits to the main branch trigger automated container rebuilds on Streamlit Cloud, ensuring continuous integration.
- **Dependency Management:** Strictly pinned dependencies in 'requirements.txt' guarantee cross-environment reproducibility.

17. Testing and Validation

The codebase includes automated unit tests implemented with Pytest in 'tests/test_core.py':

- **test_detect_issue_delivery():** Verifies that text containing 'My delivery arrived late' maps to 'Delivery'.
- **test_detect_issue_billing():** Verifies that 'I was charged twice and need a refund' maps to 'Billing & payment'.
- **test_negative_review_gets_higher_base_priority():** Confirms that negative reviews receive a higher base priority than positive reviews.
- **test_urgent_terms_raise_priority():** Verifies that urgent keywords raise priority scores above baseline negative scores.

18. Security, Privacy, and Data Governance Considerations

Data protection and operational security were core architectural considerations:

- **Zero External API Leakage:** In contrast to LLM-powered applications that stream user data to external cloud APIs, Revio InsightOps AI runs 100% of its feature extraction and inference locally within the active Python container. No data leaves the execution environment.
- **In-Memory Processing Lifecycle:** Uploaded customer reviews reside purely in transient application session state and are not persisted to disk or cloud databases, minimizing data exposure.
- **Enterprise PII Redaction Roadmap:** For enterprise production deployment, we recommend incorporating automated Personally Identifiable Information (PII) masking (e.g., using Microsoft Presidio or spaCy) to sanitize customer names, credit card numbers, and phone numbers prior to vectorization.

19. Technical Limitations

To maintain professional technical integrity, we explicitly document the following known technical boundaries:

- **1. Neutral Class Sentiment Ambiguity:** As identified during evaluation, the neutral class achieves an F1-score of 0.394 due to linguistic ambiguity and opposing sentiments in 3-star reviews.

- **2. Domain-Specific Training Corpora:** The pipeline was trained exclusively on Yelp restaurant reviews. Lexical patterns in specialized software, financial services, or B2B manufacturing may require domain retraining.
- **3. Lexical Rigidity in Rule Engine:** Keyword-based regex matching cannot detect unlisted synonyms, colloquialisms, or misspellings, defaulting unmatched reviews to 'Other'.
- **4. Uncalibrated Inference Probabilities:** Model probabilities reflect raw sigmoid/softmax outputs and have not been calibrated via Platt scaling or Isotonic regression.
- **5. Single-Label Issue Assignment:** Each review is assigned a single primary issue category based on highest match count. Multi-defect reviews do not currently support multi-label assignment.

20. Future Technical Improvements

We propose a structured four-phase engineering roadmap for subsequent iterations:

Development Phase	Target Capability	Proposed Technical Architecture
Phase 1: Calibration & Lexicon	Probability Calibration & Enhanced Rules	Implement Platt scaling / isotonic regression on classifier probabilities; expand issue regex dictionaries with synonym sets.
Phase 2: Transformer Fine-Tuning	Deep Contextual Sentiment & ABSA	Fine-tune ModernBERT or RoBERTa on domain feedback; implement Aspect-Based Sentiment Analysis (ABSA) for multi-issue reviews.
Phase 3: Semantic Topic Discovery	Unsupervised Dynamic Clustering	Integrate BERTopic / HDBSCAN embeddings to automatically discover emerging customer defect themes without manual keyword lists.
Phase 4: Grounded Enterprise LLM	Retrieval-Grounded Draft Responses & Connectors	Implement RAG-assisted response drafting with strict human approval gates; build REST connectors for Zendesk, Salesforce, and Jira.

21. Conclusion

Revio InsightOps AI validates that practical, high-impact AI engineering does not require opaque, expensive generative models. By thoughtfully pairing an n-gram TF-IDF and Logistic Regression machine learning pipeline with transparent regex issue rules and heuristic priority scoring, the system achieves fast, reliable, and auditable customer review triage on standard CPU hardware.

The project demonstrates disciplined ML engineering practices: clean data preprocessing, stratified train-test partitioning, candid evaluation reporting, automated unit testing, and modular software design. Revio InsightOps AI establishes a dependable foundation for enterprise customer feedback intelligence and operational decision support.

22. Dataset Source and Access

For complete auditability and verification, benchmark dataset access details are provided below:

- **Primary Dataset:** Yelp Restaurant Reviews Sentiment Dataset (rating_review polarity labels).
- **Curated Usable Records:** 14,350 deduplicated records (11,480 train / 2,870 test).
- **Zenodo Dataset Archive:** https://zenodo.org/records/18723813 (Public Zenodo record).

Zenodo Digital Object Identifier (DOI): https://zenodo.org/records/18723813

Google Drive Repository Access

The curated CSV files utilized in training and evaluation are mirrored on Google Drive:

Google Drive Dataset Link: real_yelp_reviews.csv - Google Drive

23. Repository and Live Application Links

Verified clickable links for technical inspection, code review, and live system testing:

Target Endpoint	Clickable Hyperlink	Technical Verification Scope
Live Streamlit Cloud Deployment	https://revio-insightops-ai.streamlit.app/	Live interactive web application for end-to-end user evaluation
GitHub Version Control Repository	https://github.com/Bhishant/revio-insightops-ai	Complete Python source code, pipeline scripts, tests, and models
Zenodo Benchmark Dataset Record	https://zenodo.org/records/18723813	Public benchmark source archive for restaurant review sentiment data
Google Drive Dataset Share	real_yelp_reviews.csv - Google Drive	Curated mirror repository for raw CSV files (replace placeholder)

- **Live Application:** https://revio-insightops-ai.streamlit.app/
- **GitHub Source Repository:** https://github.com/Bhishant/revio-insightops-ai